College Control System

وثيقة متطلبات مشروع SRS

1. مقدمة (Introduction)

1.1 الغرض (Purpose)

الغرض من هذه الوثيقة (SRS) هو تحديد المتطلبات الوظيفية وغير الوظيفية لنظام إدارة معلومات الطلاب (SIS) لكلية الهندسة بجامعة كفر الشيخ. يهدف هذا النظام إلى ميكنة (Automation) جميع العمليات الأكاديمية المتعلقة بالطالب بنظام الساعات المعتمدة، بدءاً من تسجيل المقررات وحتى التخرج، بناءً على "لائحة الساعات المعتمدة 2022" و "دليل الطالب 2025-2026".

1.2 نطاق المشروع (Scope)

النظام هو تطبيق ويب (Web Application) يخدم جميع أطراف المنظومة التعليمية (الطالب، عضو هيئة التدريس، المرشد الأكاديمي، وإدارة الكلية/الكنترول).

يشمل النطاق (In Scope):

- إدارة المستخدمين وصلاحياتهم (Students, Professors, Admins).
- إدارة البيانات الأكاديمية الأساسية (الأقسام، البرامج، المقررات، المتطلبات السابقة).
- عملية التسجيل الأكاديمي للطلاب (إضافة وحذف المقررات).
- عملية رصد الدرجات والتقديرات.
- التطبيق الآلي للائحة (حساب المعدل الفصلي والتراكمي، الإنذارات الأكاديمية، متطلبات التخرج).
- استخراج التقارير وكشوف الدرجات.
- (اختياري) نظام دعم ذكي (Chatbot) للإجابة على استفسارات اللائحة.

خارج النطاق (Out of Scope):

- الشؤون المالية (دفع المصروفات، الرسوم الدراسية).
- إدارة شؤون العاملين وأعضاء هيئة التدريس (HR System).
- أنظمة الامتحانات الإلكترونية (E-Exams).
- إدارة الجداول الدراسية (Table Management).

1.3 التعريفات والمختصرات (Definitions & Acronyms)

- **SIS:** نظام إدارة معلومات الطلاب (Student Information System).
- **اللائحة:** ملف "لائحة الساعات المعتمدة 2022". وهو المصدر الوحيد لجميع القواعد الحسابية والمنطقية.
- **الدليل:** ملف "دليل الطالب 2025-2026". وهو المصدر الوحيد للبيانات الأساسية (الأقسام، البرامج، الهيكل الإداري).
- **GPA:** المعدل التراكمي (Grade Point Average).
- **SGPA:** المعدل الفصلي (Semester GPA).
- **المتطلب السابق (Prerequisite):** مقرر يجب النجاح فيه قبل تسجيل مقرر آخر.
- **العبء الدراسي:** إجمالي عدد الساعات المعتمدة التي يسجلها الطالب في الفصل الدراسي.
- **الإنذار الأكاديمي:** حالة الطالب عندما ينخفض معدله الفصلي أو التراكمي عن حد معين (2.0 حسب اللائحة).

1.4 المراجع (References)

1. ملف: لائحة الساعات المعتمدة البكالوريوس 2022.pdf
2. ملف: دليل الطالب 2025-2026 (1).pdf

2. الوصف العام (Overall Description)

2.1 منظور المنتج (Product Perspective)

النظام هو تطبيق ويب مستقل يتم بناؤه من الصفر (New Standalone System). سيعمل كنظام مركزي لإدارة جميع بيانات الطلاب الأكاديمية، وسيوفر واجهات مختلفة لكل فئة من المستخدمين.

2.2 خصائص المستخدمين (User Characteristics)

يجب أن يدعم النظام الأدوار التالية بصلاحيات محددة:

2.2.1 الطالب (Student):

- **الهدف:** متابعة مسيرته الأكاديمية وإتمام عملية التسجيل.
- **الصلاحيات:**
  - الاطلاع على ملفه الشخصي (البيانات، القسم، البرنامج، المعدل التراكمي، الحالة الأكاديمية).
  - الاطلاع على المقررات المتاحة في الدليل واللائحة.

- تسجيل المقررات (إضافة/حذف) خلال فترة التسجيل المحددة.
- رؤية الجدول الدراسي.
- الاطلاع على درجاته وتقديراته (SGPA, CGPA).
- طباعة كشف درجات (Transcript) غير رسمي.

2.2.2 عضو هيئة التدريس (Professor):

- **الهدف:** رصد درجات الطلاب في المقررات المسندة إليه.
- **الصلاحيات:**
  - الاطلاع على المقررات المسندة إليه في الفصل الدراسي الحالي.
  - رؤية قوائم الطلاب المسجلين في مقرراته.
  - رصد الدرجات (أعمال فصلية، امتحان نهائي).
  - تصدير كشوف الرصد.

2.2.3 المرشد الأكاديمي (Academic Advisor):

- **الهدف:** توجيه الطلاب أكاديمياً واعتماد خططهم الدراسية (مذكور في المادة 5 من اللائحة).
- **الصلاحيات:**
  - (يمتلك جميع صلاحيات "عضو هيئة التدريس" +)
  - الاطلاع على الملف الأكاديمي الكامل للطلاب المسندين إليه (الدرجات، المعدلات، الإنذارات).
  - مراجعة واعتماد طلبات التسجيل المقدمة من الطلاب.
  - رفض تسجيل مقرر للطالب مع ذكر السبب (مثال: "هذا المقرر لا يناسب خطتك الدراسية").

2.2.4 مدير النظام / إدارة الكلية (Admin / Control):

- **الهدف:** الإدارة الكاملة للنظام وضمان تطبيق اللوائح.
- **الصلاحيات:**
  - إدارة المستخدمين (إضافة/تعديل/حذف جميع الأدوار).
  - إدارة البيانات الأساسية (الأقسام، البرامج، التخصصات - *تُستورد من الدليل*).
  - إدارة المقررات الدراسية (إضافة مقرر، تحديد كوده، ساعاته، متطلباته السابقة - *تُستورد من اللائحة*).
  - تحديد فترات التسجيل (بدء وانتهاء).
  - تشغيل العمليات الدورية ("محرك الكنترول").
  - استخراج التقارير الرسمية (كشوف الدرجات، شهادات التخرج، قوائم الإنذارات).

User Roles & Permissions

The system will enforce a Role-Based Access Control (RBAC) model to ensure data integrity and security. The following roles define the scope of user capabilities within the application.

Role	Primary Objective	Key Permissions
Admin	To manage the system's core academic data, user accounts, and control key academic cycles.	- Create, read, update, and delete user accounts and assign roles. - Manage academic programs, departments, and course catalogs. - Define academic semesters and create course offerings. - Control the official course registration period (open/close). - Trigger end-of-semester batch processes (GPA calculation, academic status updates). - Run graduation eligibility audits for students.
Advisor	To guide assigned students through their academic journey and approve their registration plans.	- View a list of all assigned student advisees. - Access the complete academic transcript and status of advisees. - Approve or disapprove student course registration requests. - Add academic notes or warnings to a student's profile.
Student	To manage their academic profile, register for courses, and track their progress toward graduation.	- View and edit their personal profile. - View the course catalog and academic regulations. - Register for or drop courses during the open registration period. - View their class schedule, grades, SGPA, and CGPA. - Generate and download an unofficial academic transcript. - Track progress toward meeting graduation requirements.

Instructor	To manage their assigned courses and submit final grades for enrolled students.	- View the list of courses they are assigned to teach. - Access the official student roster for each assigned course. - Submit final grades for all students in an assigned course. - Export course rosters and grade sheets.

Academic Structure Rules

The system is built upon a foundation of core academic entities.

- **Programs:** Programs (e.g., Computer Engineering, Civil Engineering, Intelligent Systems Engineering) are the primary academic tracks a student can enroll in. Each program has a defined set of required credit hours and a list of core and elective courses.
- **Courses:** The `Course` is the fundamental unit of study. Each course is defined by its key attributes: a unique Course Code, a Title, the number of Credit Hours, and the distribution of Lecture and Lab/Tutorial hours.
- **Course Coding:** All courses must adhere to the official course code structure `ABC N1N2N3` as defined in the faculty regulations.

Course Code Structure: `ABC N1N2N3`

- `ABC`: A three-letter code representing the department offering the course.
- `N1`: A single digit representing the academic level of the course (e.g., 1 for Freshman level, 2 for Sophomore, etc.).
- `N2`: A single digit representing the specific specialization track within the department.
- `N3`: A single digit representing the sequence number of the course within that specialization track.

1.3 Student Progression & Academic Status Rules

These rules govern a student's academic standing and their progression through their program of study.

- **Student Level:** A student's academic level is determined by the percentage of required credit hours they have successfully completed, as defined in **Article 20** of the regulations.

Academic Level	Percentage of Credit Hours Completed
Freshman (Level 0)	0% to < 20%
Sophomore (Level 1)	20% to < 40%
Junior (Level 2)	40% to < 60%
Senior 1 (Level 3)	60% to < 80%
Senior 2 (Level 4)	80% to 100%

- **Academic Warning & Dismissal:** The system must automatically track and apply academic standing rules based on student performance, as mandated by **Article 33**.
    - **Academic Warning:** A student receives an official academic warning if their Semester GPA (SGPA) for any primary semester is less than **2.00**.
    - **Academic Dismissal:** A student is academically dismissed from the faculty if they receive **four (4) consecutive** academic warnings. The system must explicitly track the sequence of warnings to correctly enforce this rule.

1.4 Grading and GPA Calculation Engine

The GPA calculation engine is a critical backend component that must implement the faculty's grading policies with absolute precision.

Architect's Note: Source documents present conflicting GPA scales. This design adopts the scale consistent with the course retake policy in **Article 28** (which specifies a B+ cap of 3.3 points) and the alternate conversion table in the regulations. This ensures internal consistency for all GPA calculations and will serve as the basis for initial implementation, pending final confirmation by the academic committee.

1. **Grade-to-Point Conversion:** The conversion of letter grades to grade points is based on a unified interpretation of the regulations.

Letter Grade	Grade Points
A+	4.0
A	4.0
A-	3.7
B+	3.3
B	3.0
B-	2.7
C+	2.3
C	2.0
C-	1.7
D+	1.3
D	1.0

F	0.0

1. **GPA Formulas:** The system will use the following mathematical formulas, as defined in **Article 30**, for all GPA calculations.

Semester GPA (SGPA): SGPA = Σ (Grade Points for Course × Credit Hours for Course) / Total Credit Hours Registered in Semester

Cumulative GPA (CGPA): CGPA = Σ (Grade Points for Course × Credit Hours for Course) for all semesters / Total Credit Hours Registered in all semesters

1. **GPA Exclusions:** The following grades must be strictly excluded from all SGPA and CGPA calculations:
    - **W** (Withdrawn)
    - **I** (Incomplete)
    - **P** (Pass)
2. **Course Retake Policy:** The system must implement the special rule for retaking failed courses as specified in **Article 28**. If a student retakes a compulsory course that they previously failed (received an 'F' grade), the maximum achievable grade that can be used for GPA calculation purposes is capped at **'B+' (3.3 points)**. This new grade, up to the B+ cap, completely replaces the previous failing grade in the CGPA calculation. The transcript must still show all attempts.

1.5 Course Registration & Academic Load Rules

The course registration module is governed by a set of non-negotiable constraints to ensure students follow a valid academic path.

- **Academic Load:** The maximum number of credit hours a student can register for in a semester is determined by their current CGPA, as per **Article 12** of the regulations.

Student CGPA	Maximum Credit Hours per Semester

CGPA ≥ 3.00	21
2.00 ≤ CGPA < 3.00	18
CGPA < 2.00 (Academic Warning)	14
An exception allows a graduating student to register for one additional course beyond their maximum limit to facilitate graduation.	

- **Core Validation Logic:** The registration endpoint must enforce the following rules during every registration attempt:
    1. **Prerequisite Enforcement (Article 11):** A student cannot register for a course unless they have successfully passed all of its prerequisite courses.
    2. **Credit Limit Enforcement:** A student cannot register for courses if the total credit hours for the semester exceed the maximum academic load defined by their CGPA.
    3. **No Duplicate Passes:** A student cannot register for a course they have already passed with a satisfactory grade.

1.6 Graduation Requirements

To be eligible for graduation, a student must meet all conditions summarized from **Articles 8 and 22** of the regulations. The system must provide an administrative function to audit these requirements.

1. Successfully complete the minimum required credit hours for their specific program (e.g., 172 credit hours for Intelligent Systems Engineering).
2. Achieve a final Cumulative GPA (CGPA) of **2.00 or higher**.
3. Pass both Graduation Project 1 and Graduation Project 2 with a grade of **'C' (2.0 points) or higher**.
4. Successfully complete all required Summer Training with a **'P' (Pass)** grade.

The technical design in the following section provides the blueprint for implementing these foundational business rules into a robust and reliable software system.

Key Business Logic Workflows

The following workflows illustrate how the system translates complex business rules into application logic.

- **Workflow 1: Student Course Registration (`POST /api/registrations`)**
    1. **Authentication & Authorization:** Verify the incoming request has a valid JWT and the user possesses the `Student` role.
    2. **Input Validation:** Ensure the provided `CourseOfferingId` is valid, exists, and is part of the current open registration period.
    3. **Check Prerequisites:** Query the student's academic history (`Registrations` table) to confirm that all prerequisites for the requested course have been passed with a satisfactory grade. If not, return a `400 Bad Request` with a specific error message (e.g., "Prerequisite ABC123 not met.").
    4. **Check Academic Load:** Calculate the student's new total credit hours for the semester if this course is added. Retrieve the student's CGPA and compare the new total against their maximum allowed load (Rule 1.5). If exceeded, return a `400 Bad Request`.
    5. **Check Capacity:** Atomically check if the `CourseOffering` is at full capacity. If so, return a `400 Bad Request`.
    6. **Create Registration Record:** If all checks pass, create a new `Registration` record in the database with a status of 'Pending' (if advisor approval is needed) or 'Registered'.
    7. **Return Success:** Respond with a `201 Created` status and the details of the new registration record.
- **Workflow 2: End-of-Semester GPA Calculation (`POST /api/admin/run-control-engine`)**
    1. **Authorization:** Verify the user has the `Admin` role.
    2. **Fetch Students:** Retrieve all active students who have finalized grades for the specified semester that have not yet been processed.

3. **Iterate and Calculate:** For each student, execute the following within a database transaction: a. Calculate their **SGPA** for the current semester using the formula from Rule 1.4, ensuring grades like 'W', 'I', and 'P' are excluded. b. Recalculate their **CGPA** by aggregating all graded courses from all semesters. This logic must correctly handle the course retake policy (Rule 1.4), replacing the grade points of a failed compulsory course with the new, capped grade. c. Update the `Student` record with the newly calculated SGPA and CGPA.
4. **Update Academic Status:** Based on the new SGPA, apply the Academic Warning and Dismissal logic from Rule 1.3. Update the `AcademicStatus` field and consecutive warning count on the `Student` record.
5. **Log Results:** Log the outcome of the batch process, including the number of students successfully processed and any errors encountered.
6. **Return Confirmation:** Respond with a `200 OK` and a summary report of the operation (e.g., "Processed 1500 students, 35 new academic warnings issued.").

2.3 القيود العامة (General Constraints)

- **C-1:** النظام يجب أن يلتزم **حرفياً** بجميع القواعد والمعادلات المذكورة في "لائحة الساعات المعتمدة".
- **C-2:** هيكل الأقسام والبرامج يجب أن يطابق "دليل الطالب".
- **C-3:** النظام يجب أن يكون تطبيق ويب (Web Application).
- **C-4:** لغة الواجهة **الأساسية** هي اللغة العربية.

3. المتطلبات الوظيفية (Functional Requirements)

FR-AUTH: نظام الدخول والصلاحيات

- **FR-AUTH.1:** يجب أن يوفر النظام شاشة تسجيل دخول آمنة (اسم مستخدم/رقم سري أو بريد إلكتروني/رقم سري).

- **FR-AUTH.2:** يجب أن يتعرف النظام على دور المستخدم (طالب، دكتور، ...) عند الدخول وتوجيهه للواجهة الصحيحة.
- **FR-AUTH.3:** يجب أن يمنع النظام أي مستخدم من الوصول لوظائف لا يمتلك صلاحيتها (Role-Based Access Control - RBAC).

FR-DATA: نظام إدارة البيانات الأساسية (Admin)

- **FR-DATA.1:** يجب أن يسمح النظام للـ Admin بإدارة الأقسام العلمية (إضافة/تعديل/حذف) بناءً على *الدليل* (مثل: الهندسة الميكانيكية، الهندسة الكهربية).
- **FR-DATA.2:** يجب أن يسمح النظام للـ Admin بإدارة البرامج والتخصصات داخل كل قسم (مثل: "هندسة الحاسبات والتحكم الآلي" داخل قسم "الهندسة الكهربية").
- **FR-DATA.3:** يجب أن يسمح النظام للـ Admin بإدارة بيانات الطلاب الأساسية (إضافة طالب جديد، تعديل بيانات، ربطه بقسم وبرنامج).

FR-COURSE: نظام إدارة المقررات (Admin)

- **FR-COURSE.1:** يجب أن يسمح النظام للـ Admin بإضافة/تعديل المقررات (الكود، الاسم، عدد الساعات المعتمدة، نوعه: إجباري/اختياري).
- **FR-COURSE.2:** يجب أن يسمح النظام للـ Admin بربط المقررات بمتطلباتها السابقة (Prerequisites) بناءً على جداول *اللائحة*.
- **FR-COURSE.3:** يجب أن يسمح النظام للـ Admin بطرح المقررات في فصول دراسية محددة (تحديد الأستاذ، المواعيد، السعة القصوى للطلاب).

FR-REG: نظام التسجيل (Student & Advisor)

- **FR-REG.1:** يجب أن يعرض النظام للطالب المقررات المتاحة له للتسجيل في الفصل الدراسي الحالي.
- **FR-REG.2:** يجب أن يسمح النظام للطالب بإضافة وحذف المقررات (خلال فترة التسجيل المحددة من الـ Admin فقط).
- **FR-REG.3:** يجب أن يتحقق النظام **تلقائياً** عند محاولة الإضافة من القواعد التالية (مستنبطة من *اللائحة*):
  - **[BR-PREREQ.1]** هل استوفى الطالب **المتطلبات السابقة**؟
  - **[BR-LOAD.1]** هل سيتجاوز الطالب **العبء الدراسي** المسموح له (الحد الأقصى للساعات)؟
  - **[Implicit]** هل يوجد **تعارض في المواعيد** مع مقررات أخرى مسجلة؟

- **FR-REG.4:** يجب أن يعرض النظام رسالة خطأ واضحة إذا تم خرق أي قاعدة (مثال: "لا يمكن إضافة المادة لعدم اجتياز المتطلب السابق CCE 211").
- **FR-REG.5:** يجب أن يتم إرسال طلب التسجيل للمرشد الأكاديمي (المحدد في مادة (5) من اللائحة).
- **FR-REG.6:** يجب أن يسمح النظام للمرشد بمراجعة الطلب (قبول أو رفض مع ذكر السبب).

FR-GRADE: نظام رصد الدرجات (Professor)

- **FR-GRADE.1:** يجب أن يسمح النظام للدكتور برصد درجات الطلاب (أعمال فصلية / نهائي) في مقرراته فقط.
- **FR-GRADE.2:** يجب أن يقوم النظام **تلقائياً** بحساب المجموع الكلي، وتحويله إلى تقدير (A, B+, C) وإلى نقاط (4.0, 3.7, 2.0) بناءً على **[BR-GRADE.1]**.

FR-ENGINE: محرك الكنترول (System - Admin)

- **FR-ENGINE.1:** (بعد انتهاء الرصد) يجب أن يوفر النظام للـ Admin وظيفة (مثل زر "تشغيل الكنترول للفصل الحالي").
- **FR-ENGINE.2:** عند التشغيل، يجب أن يقوم النظام بحساب الـ SGPA (المعدل الفصلي) لكل طالب بناءً على **[BR-GPA.1]**.
- **FR-ENGINE.3:** يجب أن يقوم النظام بتحديث الـ CGPA (المعدل التراكمي) لكل طالب بناءً على **[BR-GPA.1]**.

- **FR-ENGINE.4:** يجب أن يقوم النظام بتطبيق قواعد "الحالة الأكاديمية" وتحديث حالة الطالب بناءً على:
  - **[BR-STATUS.1]** (الإنذار الأكاديمي).
  - **[BR-STATUS.2]** (الفصل الأكاديمي).
- **FR-ENGINE.5:** يجب أن يقوم النظام بتحديد الطلاب الذين استوفوا **[BR-GRAD.1]** (متطلبات التخرج).

FR-REPORT: نظام التقارير

- **FR-REPORT.1:** يجب أن يسمح النظام للطالب بطباعة كشف درجات (Transcript) غير رسمي يوضح جميع المقررات، التقديرات، SGPA, CGPA.
- **FR-REPORT.2:** يجب أن يسمح النظام للـ Admin بطباعة كشف درجات رسمي.

- **FR-REPORT.3:** يجب أن يسمح النظام للـ Admin بطباعة شهادات التخرج (موضحاً بها التخصص ومعدل التخرج والتقدير العام، كما في مادة (1)).

FR-AI: (اختياري) نظام الدعم الذكي

- **FR-AI.1:** يجب أن يوفر النظام واجهة "Chatbot" (مساعد ذكي) للطلاب.
- **FR-AI.2:** يجب أن يتم تدريب الـ Chatbot **حصرياً** على محتوى ملفي "اللائحة" و "الدليل".
- **FR-AI.3:** (مثال) عند سؤال "كم ساعة يمكنني تسجيلها لو معدلي 2.5؟"، يجب أن يرد البوت: "وفقاً للمادة 12 من اللائحة، الطالب الحاصل على معدل تراكمي أقل من 3.0 يمكنه تسجيل 18 ساعة معتمدة".

4. القواعد الحاكمة (Business Rules)

(مستخرجة بالكامل من "لائحة الساعات المعتمدة 2022")

هذه هي القواعد المنطقية التي يجب أن يطبقها النظام بشكل صارم:

- **BR-GRADE.1: (من مادة 27) جدول التقديرات والنقاط:**
  - 97% فأكثر = 4.0 (A+)
  - 93% إلى أقل من 97% = 3.7 (A)
  - 89% إلى أقل من 93% = 3.3 (A-)
  - 84% إلى أقل من 89% = 3.0 (B+)
  - 80% إلى أقل من 84% = 2.7 (B)
  - 76% إلى أقل من 80% = 2.3 (B-)
  - 73% إلى أقل من 76% = 2.0 (C+)
  - 70% إلى أقل من 73% = 1.7 (C)
  - 67% إلى أقل من 70% = 1.3 (C-)
  - 64% إلى أقل من 67% = 1.0 (D+)
  - 60% إلى أقل من 64% = 0.7 (D)
  - أقل من 60% = 0.0 (F)
- **BR-GPA.1: (من مادة 30) حساب المعدل:**
  - SGPA (المعدل الفصلي) = (مجموع [نقاط المقرر × عدد الساعات المعتمدة للمقرر] لجميع مقررات الفصل) / (إجمالي الساعات المعتمدة المسجلة في الفصل)
  - CGPA (المعدل التراكمي) = (مجموع [نقاط المقرر × عدد الساعات المعتمدة للمقرر] لجميع المقررات منذ الالتحاق) / (إجمالي الساعات المعتمدة المسجلة منذ الالتحاق)
- **BR-LOAD.1: (من مادة 12) العبء الدراسي:**

- إذا كان CGPA >= 3.00: الحد الأقصى 21 ساعة معتمدة.
- إذا كان 2.00 =< و CGPA < 3.00: الحد الأقصى 18 ساعة معتمدة.
- إذا كان CGPA < 2.00 (إنذار): الحد الأقصى 14 ساعة معتمدة.
- استثناء: يمكن للطالب المتوقع تخرجه تسجيل مقرر إضافي واحد عن الحد الأقصى.

- **BR-STATUS.1: (من مادة 33) الإنذار الأكاديمي:**
  - "يحصل الطالب على إنذارًا أكاديميًا إذا كان معدله الفصلي في أي فصل دراسي رئيسي أقل من 2.00."
- **BR-STATUS.2: (من مادة 33) الفصل الأكاديمي:**
  - "يفصل الطالب إذا حصل الطالب على أربعة إنذارات أكاديمية متتابعة." (يجب على النظام تتبع عدد الإنذارات المتتابعة).
- **BR-PREREQ.1: (من مادة 11) المتطلب السابق:**
  - "لا يجوز تسجيل الطالب في أي مقرر دراسي يشترط له متطلب سابق ما لم يكن قد اجتاز بنجاح ذلك المتطلب." (النظام يجب أن يمنع التسجيل).
- **BR-ATTENDANCE.1: (من مادة 19) الحرمان بسبب الغياب:**
  - "عند تجاوز نسبة الغياب 25%... يحرم الطالب من أداء الامتحان النهائي... ويرصد له التقدير (F)." (يجب أن يكون هناك خانة للدكتور لتحديد "محروم").
- **BR-GRAD.1: (من مادة 8 و 22) متطلبات التخرج:**
  - إكمال الحد الأدنى من الساعات المعتمدة للبرنامج (مثال: 172 ساعة لهندسة النظم الذكية).
  - الحصول على معدل تراكمي CGPA >= 2.00.
  - النجاح في مشروع التخرج (1 و 2) بتقدير لا يقل عن C (أو 2.0 نقطة) لكل منهما. (مذكورة في مادة 22 ومادة 8).
  - أداء التدريب الصيفي (3 تدريبات) بنجاح (تقدير P - Pass). (مذكورة في مادة 21).

5. متطلبات البيانات (Data Requirements)

(الكيانات الأساسية المطلوبة لقاعدة البيانات)

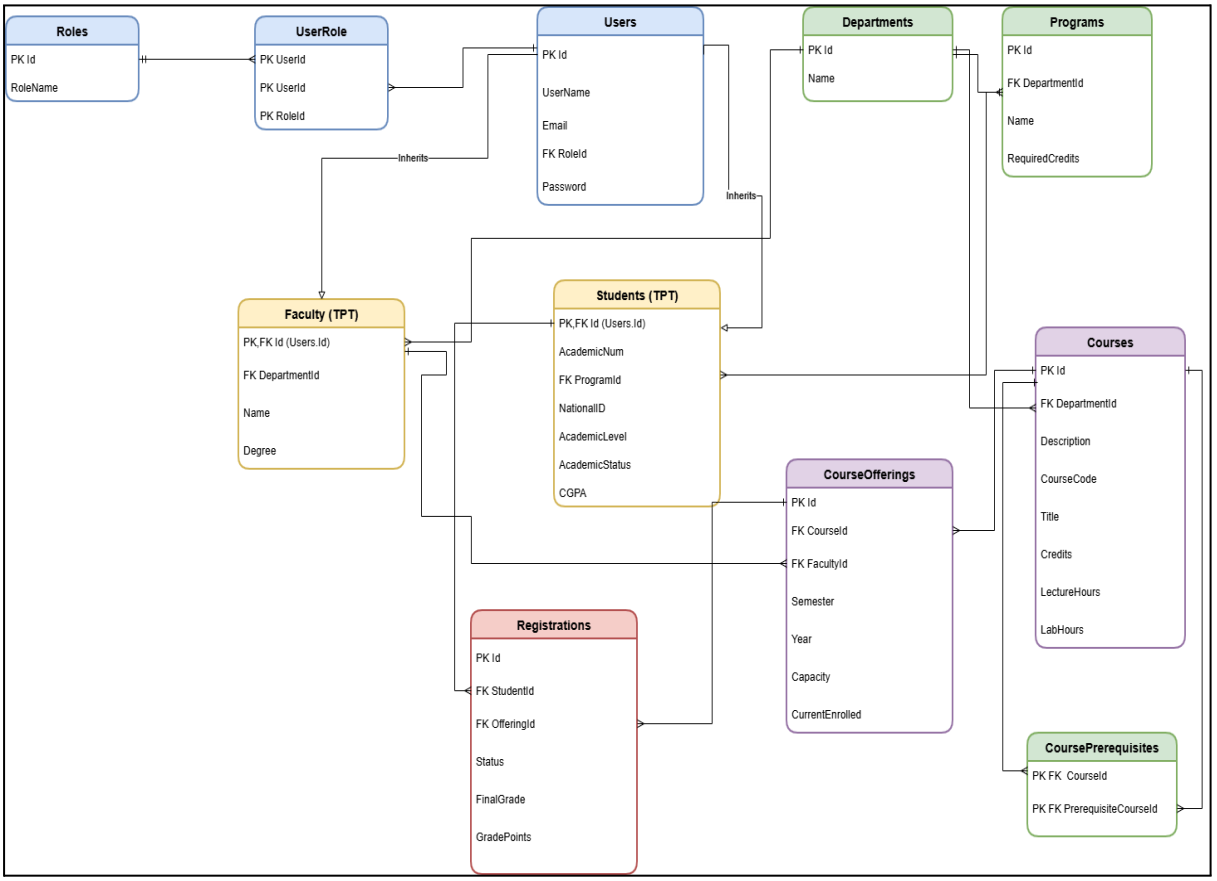
- **DR-STUDENT:** (الرقم الأكاديمي [PK]، الاسم، الرقم القومي، القسم [FK]، البرنامج [FK]، المستوى، الحالة الأكاديمية [CGPA، FK]، البريد الإلكتروني، ...).
- **DR-FACULTY:** (الرقم الوظيفي [PK]، الاسم، القسم [FK]، الدرجة العلمية، الدور [FK: دكتور/مرشد]، البريد الإلكتروني، ...).
- **DR-DEPARTMENT:** (كود القسم [PK]، اسم القسم).
- **DR-PROGRAM:** (كود البرنامج [PK]، اسم البرنامج، القسم [FK]).

- **DR-COURSE:** (كود المقرر [PK]، اسم المقرر، عدد الساعات المعتمدة، المستوى، القسم [FK]، الوصف).
- **DR-PREREQUISITE:** (جدول يربط المقررات بمتطلباتها: CourseID [FK], PrerequisiteCourseID [FK]).
- **DR-SEMESTER_COURSE:** (جدول المقررات المطروحة: SemesterID [FK], CourseID [FK], FacultyID [FK], MaxCapacity, CurrentCapacity, Timings).
- **DR-REGISTRATION:** (جدول تسجيل الطالب: StudentID [FK], SemesterCourseID [FK], Status [Pending, Approved, Dropped]).
- **DR-GRADE:** (جدول الدرجات: RegistrationID [PK/FK], SemesterWorkGrade, FinalGrade, TotalGrade, LetterGrade, GradePoints).

6. المتطلبات غير الوظيفية (Non-Functional Requirements)

- **NFR-SEC.1 (الأمان):** جميع الاتصالات بين المتصفح والسيرفر يجب أن تكون مشفرة (HTTPS).
- **NFR-SEC.2 (الأمان):** كلمات المرور يجب أن تكون مُجزأة (Hashed) ولا تُخزن كنص صريح.
- **NFR-PERF.1 (الأداء):** النظام (خاصة شاشة التسجيل) يجب أن يتحمل ضغط 1000 مستخدم متزامن (Concurrent Users) خلال أسبوع التسجيل.
- **NFR-PERF.2 (الأداء):** يجب ألا يستغرق تشغيل "محرك الكنترول" (FR-ENGINE) أكثر من 5 دقائق لجميع طلاب الكلية.
- **NFR-USE.1 (سهولة الاستخدام):** الواجهة يجب أن تكون باللغة العربية، واضحة، وسهلة الاستخدام.
- **NFR-USE.2 (سهولة الاستخدام):** النظام يجب أن يكون متوافقاً مع المتصفحات الرئيسية (Chrome, Firefox, Safari).
- **NFR-USE.3 (سهولة الاستخدام):** النظام يجب أن يكون متجاوباً (Responsive) للعمل بكفاءة على شاشات الموبايل.
- **NFR-MAINT.1 (الصيانة):** الكود (خاصة الـ Backend) يجب أن يكون موثقاً جيداً (Code Comments) ليسهل على فريق آخر صيانته وتطويره.
- **NFR-RELIAB.1 (الموثوقية):** يجب أن يتضمن النظام آلية للنسخ الاحتياطي (Backup) لقاعدة البيانات بشكل دوري (يومي).

DB Design



✅ 1) ROLES

==========

📘 Concept

Represents system roles used in **RBAC (Role-Based Access Control)**.

The SRS defines four roles:

- Admin
- Student
- Advisor
- Instructor

🧩 Properties

Property	Type	Meaning
RoleID	PK	Unique identifier
Name	text	The name of the system role (Admin, Student...)

🔗 Relationships

- **1-to-many** with UserRole
- **Indirect** relationship to Users through UserRole

🛠️ SQL (PostgreSQL)

sql Copy code

```sql
CREATE TABLE Roles (
    RoleID SERIAL PRIMARY KEY,
    Name VARCHAR(50) NOT NULL UNIQUE
);
```

📝 Example INSERT

sql Copy code

```sql
INSERT INTO Roles (Name)
VALUES ('Admin'), ('Student'), ('Advisor'), ('Instructor');
```

🎯 Real Scenario Example

A student logs in → system checks if their `Role = Student` to decide what UI they see.

Activat

==========

✅ 2) USERROLE (Mapping Table)

==========

📘 Concept

Because one user can technically have multiple roles (e.g., Instructor + Advisor), this table acts as a **bridge** between Users and Roles.

🧩 Properties

Property	Meaning
UserID	FK → Users
RoleID	FK → Roles

🔗 Relationships

- Many-to-many link between **Users** and **Roles**

🛠️ SQL

sql Copy code

```sql
CREATE TABLE UserRole (
    UserID INT NOT NULL,
    RoleID INT NOT NULL,
    PRIMARY KEY (UserID, RoleID),
    FOREIGN KEY (UserID) REFERENCES Users(UserID),
    FOREIGN KEY (RoleID) REFERENCES Roles(RoleID)
);
```

📝 Example INSERT

sql Copy code

```sql
INSERT INTO UserRole (UserID, RoleID)
VALUES (1, 2); -- Assign Student role to User 1
```

🎯 Real Scenario

A professor who is also an advisor:

2 rows appear for the same UserID.

✅ 3) USERS

==========

📘 Concept

Master identity table.

Faculty + Students inherit ("TPT") from Users.

🧩 Properties

Property	Meaning
UserID	PK
FirstName	User first name
LastName	User last name
Email	Unique login
PasswordHash	Hashed password
Phone	optional contact

🔗 Relationships

- 1-to-many with UserRole
- 1-to-1 with Students (TPT)
- 1-to-1 with Faculty (TPT)

🛠 SQL

```sql
CREATE TABLE Users (
    UserID SERIAL PRIMARY KEY,
    FirstName VARCHAR(100) NOT NULL,
    LastName VARCHAR(100) NOT NULL,
    Email VARCHAR(150) NOT NULL UNIQUE,
    PasswordHash VARCHAR(255) NOT NULL,
    Phone VARCHAR(30)
);
```

📝 Example INSERT

```sql
INSERT INTO Users (FirstName, LastName, Email, PasswordHash)
VALUES ('Ahmed', 'Hassan', 'ahmed@uni.edu', 'HASHED-PW-123');
```

🎯 Real Scenario

This row represents any human using the system.

✅ 4) DEPARTMENTS

==========

📘 Concept

Each academic unit in the faculty (CS, Electrical, Mechanical...).

🧩 Properties

Property	Meaning
DepartmentID	PK
DepartmentName	Official name
Description	optional

🔗 Relationships

- 1-to-many with Programs
- 1-to-many with Faculty
- 1-to-many with Courses

🛠️ SQL

```sql
CREATE TABLE Departments (
    DepartmentID SERIAL PRIMARY KEY,
    DepartmentName VARCHAR(100) NOT NULL UNIQUE,
    Description TEXT
);
```

📝 Example INSERT

```sql
INSERT INTO Departments (DepartmentName)
VALUES ('Computer Engineering'), ('Electrical Engineering');
```

🎯 Real Scenario

Courses + students are tied to a department through programs.

==========

✅ 5) PROGRAMS

==========

📘 Concept

Academic study track (e.g., Intelligent Systems Engineering).

🧩 Properties

Property	Meaning
ProgramID	PK
ProgramName	Name of the program
DepartmentID	FK → Department
RequiredCredits	Credits needed for graduation

🔗 Relationships

- Many students belong to 1 program
- Many courses can be linked logically but not enforced here

🛠️ SQL

```sql
CREATE TABLE Programs (
    ProgramID SERIAL PRIMARY KEY,
    ProgramName VARCHAR(150) NOT NULL UNIQUE,
    DepartmentID INT NOT NULL,
    RequiredCredits INT NOT NULL,
    FOREIGN KEY (DepartmentID) REFERENCES Departments(DepartmentID)
);
```

📝 Example INSERT

```sql
INSERT INTO Programs (ProgramName, DepartmentID, RequiredCredits)
VALUES ('Intelligent Systems Engineering', 1, 172);
```

🎯 Scenario

A student's graduation audit pulls RequiredCredits from this table.

✅ 6) FACULTY (TPT – inherits Users)

==========

📘 Concept

Subset of Users: professors, advisors, instructors.

🧩 Properties

Property	Meaning
FacultyID	PK
UserID	FK → Users
DepartmentID	FK
AcademicRank	Lecturer, Professor, Assistant Prof...

🔗 Relationships

- 1-to-1 with Users
- 1-to-many with CourseOfferings (instructor teaches offering)
- 1-to-many with Students (advisor)

🛠️ SQL

```sql
CREATE TABLE Faculty (
    FacultyID SERIAL PRIMARY KEY,
    UserID INT NOT NULL UNIQUE,
    DepartmentID INT NOT NULL,
    AcademicRank VARCHAR(100),
    FOREIGN KEY (UserID) REFERENCES Users(UserID),
    FOREIGN KEY (DepartmentID) REFERENCES Departments(DepartmentID)
);
```

📝 Example INSERT

```sql
INSERT INTO Faculty (UserID, DepartmentID, AcademicRank)
VALUES (5, 1, 'Associate Professor');
```

🎯 Scenario

Used when assigning an instructor to a course.

==========

✅ 7) STUDENTS (TPT – inherits Users)

==========

📘 Concept

Student-specific record extending Users.

🧩 Properties

Property	Meaning
StudentID	PK
UserID	FK → Users
AcademicNumber	Official student code
ProgramID	FK
AdvisorID	FK → Faculty
CGPA	Cumulative GPA
AcademicStatus	Good, Warning, Dismissed

🔗 Relationships

- 1-to-many with Registrations
- 1-to-1 with Users

🛠️ SQL

```sql
CREATE TABLE Students (
    StudentID SERIAL PRIMARY KEY,
    UserID INT NOT NULL UNIQUE,
    AcademicNumber VARCHAR(20) NOT NULL UNIQUE,
    ProgramID INT NOT NULL,
    AdvisorID INT,
    CGPA DECIMAL(3,2) DEFAULT 0,
    AcademicStatus VARCHAR(50) DEFAULT 'Good Standing',
    FOREIGN KEY (UserID) REFERENCES Users(UserID),
    FOREIGN KEY (ProgramID) REFERENCES Programs(ProgramID),
    FOREIGN KEY (AdvisorID) REFERENCES Faculty(FacultyID)
);
```

📝 Example INSERT

```sql
INSERT INTO Students (UserID, AcademicNumber, ProgramID)
VALUES (4, '202500123', 1);
```

🎯 Scenario

Used for: transcript, GPA, registration eligibility.

==========

✅ 8) COURSES

==========

📘 Concept

Every subject in the faculty's course catalog.

🧩 Properties

Property	Meaning
CourseID	PK
Code	Official 6-character code (ABC123)
Title	Course name
DepartmentID	FK
Level	1–4 (Freshman → Senior)
Credits	Credit hours
LectureHours	Hours/week
LabHours	Hours/week
Description	optional

🔗 Relationships

- 1-to-many with CourseOfferings
- many-to-many with itself (prerequisites)

🛠️ SQL

```sql
CREATE TABLE Courses (
    CourseID SERIAL PRIMARY KEY,
    Code VARCHAR(10) NOT NULL UNIQUE,
    Title VARCHAR(255) NOT NULL,
    DepartmentID INT NOT NULL,
    Level INT NOT NULL,
    Credits INT NOT NULL,
    LectureHours INT NOT NULL,
    LabHours INT NOT NULL,
    Description TEXT,
    FOREIGN KEY (DepartmentID) REFERENCES Departments(DepartmentID)
);
```

📝 Example INSERT

```sql
INSERT INTO Courses
(Code, Title, DepartmentID, Level, Credits, LectureHours, LabHours)
VALUES ('CSE211', 'Data Structures', 1, 2, 3, 2, 2);
```

🎯 Scenario

Used in registration, prerequisites, GPA, transcript.

==========

✅ 9) COURSEPREREQUISITES (Self Many-to-Many)

==========

📘 Concept

Connects a course with all required courses that must be passed first.

🧩 Properties

Property	Meaning
CourseID	course being registered
PrerequisiteCourseID	required course

🔗 Relationships

- many-to-many between Courses

🔗 Relationships

- many-to-many between Courses

🛠️ SQL

```sql
CREATE TABLE CoursePrerequisites (
    CourseID INT NOT NULL,
    PrerequisiteCourseID INT NOT NULL,
    PRIMARY KEY (CourseID, PrerequisiteCourseID),
    FOREIGN KEY (CourseID) REFERENCES Courses(CourseID),
    FOREIGN KEY (PrerequisiteCourseID) REFERENCES Courses(CourseID)
);
```

📝 Example

```sql
INSERT INTO CoursePrerequisites
VALUES (10, 3); -- Course 10 requires passing Course 3
```

🎯 Scenario

Registration engine enforces BR-PREREQ.

==========

✅ 10) COURSEOFFERINGS

==========

📘 Concept

Represents a course taught in a specific semester with an instructor and capacity.

🧩 Properties

Property	Meaning
OfferingID	PK
CourseID	FK
InstructorID	FK → Faculty
Semester	Fall/Spring/Summer
Year	academic year
MaxCapacity	seats available
CurrentCapacity	seats currently filled
Schedule	text (day/time)

🔗 Relationships

- 1-to-many with Registrations
- many offerings belong to 1 course

🔗 Relationships

- 1-to-many with Registrations
- many offerings belong to 1 course

🛠️ SQL

sql Copy code

```sql
CREATE TABLE CourseOfferings (
    OfferingID SERIAL PRIMARY KEY,
    CourseID INT NOT NULL,
    InstructorID INT NOT NULL,
    Semester VARCHAR(20) NOT NULL,
    Year INT NOT NULL,
    MaxCapacity INT NOT NULL,
    CurrentCapacity INT DEFAULT 0,
    Schedule VARCHAR(100),
    FOREIGN KEY (CourseID) REFERENCES Courses(CourseID),
    FOREIGN KEY (InstructorID) REFERENCES Faculty(FacultyID)
);
```

📝 Example INSERT

sql Copy code

```sql
INSERT INTO CourseOfferings
(CourseID, InstructorID, Semester, Year, MaxCapacity)
VALUES (10, 3, 'Fall', 2025, 40);
```

🎯 Scenario

Used when a student registers—they register for an *offering*, not the course itself.

==========

✅ 11) REGISTRATIONS

==========

📘 Concept

Represents each attempt of a student taking a course offering.
This is where grades & status are stored.

🧩 Properties

Property	Meaning
RegistrationID	PK
StudentID	FK
OfferingID	FK
Status	Pending, Approved, Dropped, Completed
SemesterWorkGrade	numeric
FinalExamGrade	numeric
TotalGrade	numeric
LetterGrade	A, B+, C...
GradePoints	numeric for GPA

🔗 Relationships

- many-to-1 with Students
- many-to-1 with CourseOfferings

🛠 SQL

sql Copy code

```sql
CREATE TABLE Registrations (
    RegistrationID SERIAL PRIMARY KEY,
    StudentID INT NOT NULL,
    OfferingID INT NOT NULL,
    Status VARCHAR(20) DEFAULT 'Pending',
    SemesterWorkGrade DECIMAL(5,2),
    FinalExamGrade DECIMAL(5,2),
    TotalGrade DECIMAL(5,2),
    LetterGrade VARCHAR(3),
    GradePoints DECIMAL(3,2),
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID),
    FOREIGN KEY (OfferingID) REFERENCES CourseOfferings(OfferingID)
);
```

📝 Insert Example

sql Copy code

```sql
INSERT INTO Registrations (StudentID, OfferingID, Status)
VALUES (12, 5, 'Pending');
```

🎯 Scenario

After the instructor submits grades, GPA engine calculates SGPA/CGPA using this table.

9 DR-GRADE

🔥 درجات الطالب في المادة

المفتاح الأساسي = المفتاح الأجنبي
لأنه One-to-One مع التسجيل.

العمود	معناه
RegistrationID (PK/FK)	نفس رقم التسجيل
SemesterWorkGrade	أعمال السنة
FinalGrade	النهائي
TotalGrade	المحصلة
LetterGrade	A — B+ — C...
GradePoints	نقاط GPA

🎓 الفرق بين Course و CourseOffering

✅ Course = المادة نفسها (تعريف ثابت)

زي:

- الكود CSE211
- الاسم Data Structures
- عدد الساعات
- المتطلبات السابقة

لا يتغير من فصل لفصل.

✅ CourseOffering = طرح المادة في فصل معين

هو نسخة من الـCourse **مرتبطة بفصل معين + دكتور معين + جدول + سعة.**

مثلاً نفس المادة CSE211 ممكن تتطرح كالتالي:

- Fall 2025 مع Dr. Ahmed سعة 40
- Fall 2025 مع Dr. Mona سعة 35
- Spring 2026 مع Dr. Ali سعة 45

🎯 لماذا لا نستخدم Course فقط؟

لأن Course ثابت
لكن الطالب يسجل في المادة **في فصل معين** بظروف مختلفة.

بدون CourseOffering لن تستطيع معرفة:

- الطالب أخذ المادة في أي فصل؟
- مع أي Doctor؟
- في أي شعبة (Group)؟
- مواعيد المحاضرات؟
- سعة الشعبة؟
- محاولة الطالب رقم كام؟ (للتعثر/الإعادة)

- **حوار ال Delete**

📋 كل Tables في مشروعك

🟦 Academic Structure (تعريفية)

Department

- Delete ❌
- Deactivate ✅

ليه؟ مكتوب في الشهادات

Program

- Delete ❌
- Deactivate ✅

ليه؟ خطة دراسة وطلاب قدام

Course

- Delete ❌
- Deactivate ✅

ليه؟ درجات + GPA

Semester

- Delete ❌
- Deactivate ✅

ليه؟ دورة زمنية رسمية

🟩 Actors (أشخاص)

Student

- Delete ❌
- Deactivate ❌
- **Status** ✅
  - Active
  - Graduated
  - Dismissed

Faculty

- Delete ❌
- Deactivate ❌
- **Status** ✅
  - Working
  - Retired

AppUser

- Delete ❌ (بعد ما ليه بيانات)
- Disable / Lock ✅

Transactional / History

Registration

- Delete ❌
- Deactivate ❌
- Status ❌
- ✅ سيبه تاريخ

Grade

- Delete ❌
- Deactivate ❌
- ✅ سيبه تاريخ

Transcript / GPA

- Delete ❌
- Deactivate ❌
- ✅ تاريخ رسمي

Admin / System (ملوش تاريخ)

Role

- Delete ✅

Permissions

- Delete ✅

Dummy / Test Data

- Delete ✅

📋 جدول ملخّص نهائي

Table	Delete	Deactivate	Status
Department	❌	✅	❌
Program	❌	✅	❌
Course	❌	✅	❌
Semester	❌	✅	❌
Student	❌	❌	✅
Faculty	❌	❌	✅
AppUser	❌	❌	Disable
Registration	❌	❌	❌
Grade	❌	❌	❌
Role	✅	❌	❌

❓ هل كل مرة أجيب Entity أشيّك Status أو IsActive؟

لا ❌

لو عملتها كده هتتعب وهتنسى checks.

الحل الصح: **النظام يشيل عنك الشغل ده تلقائيًا.**

1️⃣ الفرق الأول (مهم)

- **IsActive / Deactivate**
  → خاص بالـ *كيانات التعريفية*
  (Department – Program – Course)
- **Status**
  → خاص بالـ *الأشخاص*
  (Student – Faculty)

2️⃣ إزاي أتعامل مع IsActive بدون تعب؟

✅ الحل الصح: Global Query Filter (EF Core)

مثال

Copy code csharp

```csharp
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Department>()
        .HasQueryFilter(d => d.IsActive);

    modelBuilder.Entity<Program>()
        .HasQueryFilter(p => p.IsActive);

    modelBuilder.Entity<Course>()
        .HasQueryFilter(c => c.IsActive);
}
```

النتيجة

- أي Query عادي:

Copy code csharp

```csharp
_context.Departments.ToList();
```

النتيجة

- أي Query عادي:

Copy code csharp

```csharp
_context.Departments.ToList();
```

➡️ يرجع Active بس

- مش محتاج تكتب:

Copy code csharp

```csharp
Where(x => x.IsActive)
```

كل مرة ❌

🟡 طب لو محتاج أجيب الكل؟ (تقارير)

Copy code csharp

```csharp
_context.Departments
    .IgnoreQueryFilters()
    .ToList();
```

---

3 طب والـ Status بتاعة Student و Faculty؟

Status مش Filter تلقائي

ليه؟

- لأن:
  - طالب متخرج → لسه مهم
  - طالب مفصول → لسه مهم

التعامل الصح

- تعمل checks بس في Use Cases محددة

🟢 مثال: تسجيل طالب مادة

Copy code 🗐 csharp

```csharp
if (student.Status != StudentStatus.Active)
    return Result.Failure(StudentErrors.NotActive);
```

🟢 مثال: رصد درجات

Copy code 🗐 csharp

```csharp
if (faculty.Status != FacultyStatus.Working)
    return Result.Failure(FacultyErrors.NotWorking);
```

📌 مش كل Query

بس وقت الفعل المهم

Backend Implementation

- **Domain Layer**

Folder Structure Overview

Plaintext

```
src/CollegeControl.Domain/
├── Abstractions/            # Base DDD Classes
├── Shared/                  # Common Value Objects (Grade, Semester)
├── Departments/             # Departments & Programs Aggregate
├── Users/                   # User Base Aggregate
├── Faculty/                 # Faculty Aggregate (Inherits User)
├── Students/                # Student Aggregate (Inherits User)
├── Courses/                 # Course Aggregate
├── CourseOfferings/         # CourseOffering Aggregate
└── Registrations/           # Registration Aggregate
```

- https://gemini.google.com/share/bc8a8614c1ae

✅ تلخيص ApplyRetakeCap()

لو الطالب **أعاد مادة**
وجاب درجة **أعلى من** B+ (يعني نقاطها < 3.3)

→ النظام **يخفضها ويخليها** B+

ليه؟
عشان لائحة الكلية:

> "أعلى درجة في الإعادة = B+ مهما كانت درجتك."

✅ تلخيص IsIncludedInGpa()

بعض الدرجات **ما تتحسبش في المعدل** GPA وهي:

- **W** → انسحاب (Withdrawn)
- **I** → غير مكتمل (Incomplete)
- **P** → نجاح بدون نقاط (Pass)

أي درجة غير دول → **تتحسب في المعدل** GPA.

- In course

Regex Pattern 1

Copy code

```
private static readonly Regex CodePattern = new(@"^([A-Z]{3})\s?([0-9])([0-9])([0-9])$", Regex
```

معناه:

- `([A-Z]{3})` → أول 3 حروف Capital (قسم الكلية)
  مثل: `CS` / `CCE` (بس هنا 3 حروف فقط)
- `\s?` → ممكن يكون في space أو لأ
- `([0-9])([0-9])([0-9])` → ثلاث أرقام
  مثل: `101`

يعني pattern يقبل:

- `CCE101`
- `CCE 101`

Properties 2

Copy code

```
public string Value { get; }
public string DepartmentPart { get; }
public int LevelPart { get; }
```

- **Value** → الكود كامل بعد التنسيق "CCE 101"
- **DepartmentPart** → أول 3 حروف "CCE"
- **LevelPart** → أول رقم من الكود "1"

🎓 إيه هي Credits؟

Credits = عدد الساعات/النقاط اللي بتمثّل وزن المادة

بمعنى:

- كل مادة في الجامعة ليها **وزن أكاديمي**
- الوزن دا بيتقاس بحاجة اسمها **Credit Hours**
- عدد الكريديتس هو اللي بيحدد:
- المادة تقيلة ولا خفيفة؟
- الطالب يقدر ياخد كام مادة في الترم؟
- إزاي نحسب الـ GPA ؟

🔍 مثال بسيط

لو مادة **Programming 1** فيها:

- 3 ساعات محاضرة في الأسبوع
- 2 ساعة معمل Lab

بيبقى عدد الـ **Credits** غالبًا = 3
(لأن معظم الجامعات بتعتبر الكريديت = ساعة محاضرة)

ومادة Lab تقيلة ممكن تبقى:

- 1 ساعة محاضرة + 3 ساعات Lab
  → credits = 2

- ف student domain model عشان نحسب أقصى عدد ساعات دراسية (credit hours) اللي الطالب مسموح له يسجلها في الفصل الدراسي محتاج شويه ال rules دا الخلاصه

مثال عملي

CGPA	AcademicStatus	Graduating	Max Credits
3.5	GoodStanding	False	21
2.5	GoodStanding	False	18
1.8	AcademicWarning	False	14
3.2	GoodStanding	True	24 (21+3)

- ف student domain model فيه method اسمها ProcessSemesterResults
  - الوظيفة الرئيسية: تحديث حالة الطالب الأكاديمية بعد نهاية كل فصل دراسي حسب نتائج الـ semester (SGPA) والمعدل التراكمي (CGPA).

مثال عملي

SGPA الفصل	CGPA الكلي	الحالة السابقة	ConsecutiveWarnings	النتيجة بعد العملية
1.8	2.5	GoodStanding	0	AcademicWarning, 1
1.9	2.4	AcademicWarning	1	AcademicWarning, 2
2.5	2.6	AcademicWarning	2	GoodStanding, 0
1.5	1.9	AcademicWarning	3	Dismissed

- **Application Layer**
  - Don’t forget to solve circular loops between creating user and creating student or faculty in registerCommandHandler

12-Week Roadmap & Sprint Plan

📌 WEEK 1 — Solution Setup + Core Architecture Skeleton

🎯 Goals

Set up the entire project structure, architecture boundaries, and core infrastructure to enable development.

📦 Deliverables

- Clean Architecture folder structure (Api, Application, Domain, Infrastructure, SharedKernel)
- Project references set correctly
- Dependency rules enforced
- Docker-based Postgres ready
- Global exception handling
- Logging (Serilog)
- Swagger UI

🛠 Tasks

- Create solution + projects
- Setup Clean Architecture folder tree
- Setup DI container in API
- Configure `Serilog`, `Swagger`, `CORS`, `HTTPS`
- Create Docker Compose: API + Postgres + pgAdmin/Adminer
- Setup basic EF Core DbContext (empty)
- Create SharedKernel (Result<T>, DomainExcepti ↓ ValueObjects folder)

🧪 Testing

- API health check endpoint
- Verify database connection
- CI pipeline: build + restore

📝 Documentation

- Project structure explanation
- How to run with Docker
- Architectural decisions record (ADR #1)

📌 WEEK 2 — Domain Layer (DDD Core)

🎯 Goals

Build the entire **Domain Layer** (aggregates, entities, value objects, domain events).

📦 Deliverables

- Complete domain model for:
  - Users, Roles, UserRole
  - Students, Faculty
  - Departments, Programs
  - Courses, CoursePrerequisites
  - CourseOffering
  - Registrations
  - GraduationProject
- Domain Events defined
- Aggregates validated with invariants
- Value Objects implemented

🛠️ Tasks

- Implement base abstractions (Entity, DomainEvent)
- Implement value objects (CourseCode, Grade, AcademicStatus)
- Implement aggregates (Student, Course, CourseOffering)
- Add domain behaviors (AddRegistration, Increm Capacity, SubmitGrades, etc.)
- Add domain events where relevant

Activate W

🧪 Testing

- Unit tests for aggregates (prereq validation, capacity rules, CGPA update, etc.)
- Test domain events fire correctly

📝 Documentation

- Domain Model Document
- Aggregate boundaries explanations
- Invariant rules

📌 WEEK 3 — Infrastructure Setup + EF Core Modeling

🎯 Goals

Implement infrastructure layer with full EF Core mapping.

📦 Deliverables

- EF Core DbContext with:
  - TPT mapping for User → Student / Faculty
  - Tables for all aggregates
  - Fluent API configurations
- Repository interfaces implemented
- Database migrations created

🛠️ Tasks

- Configure EF Core + Npgsql provider
- Implement TPT for Students/Faculty
- Implement all entity configurations
- Setup repositories:
  - `IStudentRepository`
  - `ICourseRepository`
  - `ICourseOfferingRepository`
  - `IRegistrationRepository`
- Apply first migration
- Seed reference data (roles, departments, programs)

🧪 Testing

- Integration tests for repositories
- Migration test in CI pipeline

📝 Documentation

- Database schema diagram
- Mapping strategies (TPT explanation)

📌 WEEK 4 — Authentication & Authorization Layer

🎯 Goals

Implement Identity + JWT security.

📦 Deliverables

- Custom user identity integrated with your Users table
- JWT auth configured
- Role-based authorization policies
- Authentication endpoints (Login, Refresh, Logout)

🛠 Tasks

- Implement user creation logic
- Map Roles to Claims roles
- Hash passwords using Identity's PasswordHasher
- Add middleware for JWT auth
- Implement:
  - `POST /auth/login`
  - `POST /auth/refresh`
  - `POST /auth/register` (Admin only)
- Seed Admin user + Admin role

🧪 Testing

- Integration tests for:
  - Login success
  - Invalid credentials
  - Role-based permissions

📝 Documentation

- Auth architecture flow
- JWT lifetime strategy
- Role permissions table

📌 WEEK 5 — Admin Module (Departments, Programs, Courses)

🎯 Goals

Build full CRUD management for academic data.

📦 Deliverables

- CRUD endpoints for:
  - Departments
  - Programs
  - Courses
  - Prerequisites
- Validations (unique course code, valid credit hours, etc.)

🛠 Tasks

- Commands (AddDepartment, AddCourse, UpdateCourse, etc.)
- Queries (GetDepartmentList, GetCourseDetails, etc.)
- Validators (FluentValidation)
- Automapper profiles

🧪 Testing

- Full API integration tests for admin endpoints
- Domain-level validation tests

📝 Documentation

- Admin module API documentation
- Course model explanation

📌 WEEK 6 — Course Offering Module

🎯 Goals

Instructors and admins can create course offerings.

📦 Deliverables

- API endpoints:
  - Create offering
  - Update schedule, capacity
  - List offerings per course / instructor
- Domain events when offering created

🛠 Tasks

- Add Offering aggregate logic
- Offerings Commands (CreateOffering, UpdateOffering)
- Offerings Queries (GetOfferingsForCourse)
- Add schedule conflict validation

🧪 Testing

- Test schedule overlaps
- Test capacity updates

📝 Documentation

- Offering creation workflow

📌 WEEK 7 — Student Registration Module (Core Feature)

🎯 Goals

Implement complex registration rules.

📦 Deliverables

- Student registration flow with:
  - Prereq validation
  - Offering capacity validation
  - Academic status rules
  - Max credit load per semester
- Pending, Approved, Withdrawn statuses

🛠 Tasks

- Command: `RegisterStudentToOfferingCommand`
- Handler performs:
  - Fetch student + offering
  - Check prereqs
  - Check capacity
  - Validate academic status
- Add domain events (`StudentRegisteredToCourse`)
- Implement `WithdrawRegistrationCommand`

🧪 Testing

- Integration tests for registration edge cases
- Domain tests: cannot register twice

📝 Documentation

- Registration rulebook
- Validation flow diagram

📌 WEEK 8 — Advisor Workflow

🎯 Goals

Advisor approval system for registrations.

📦 Deliverables

- Advisor-specific endpoints:
  - View advisees
  - Approve/Reject pending registrations
  - Lock/unlock registration
- Advisor assignment logic

🛠 Tasks

- Advisor Queries (GetAdvisees)
- Registration approval commands
- AdvisorAssignment in Student aggregate
- Advisor permissions enforcement

🧪 Testing

- Ensure non-advisors cannot approve
- Validate incorrect student/advisor mismatches

📝 Documentation

- Advisor workflow diagram
- Approval process rules

📌 WEEK 9 — Instructor Grading Module

🎯 Goals

Enable faculty to submit grades.

📦 Deliverables

- Submit semester work & final exam
- Assign letter grade (A+, A, A-, ...)
- Calculate grade points
- GradeRecord aggregate or reuse Registration
- Instructor restrictions (only their offering)

🛠 Tasks

- `SubmitGradesCommand`
- Domain event: `GradesSubmittedDomainEvent`
- Logic for grade mapping
- Validate max score per component

🧪 Testing

- Edge cases: invalid grade, exceeding score
- Permissions: other instructors shouldn't grade

📝 Documentation

- Grading rules & mapping table

📌 WEEK 10 — GPA Engine & Academic Status Update (Control Engine)

🎯 Goals

Automate calculation of SGPA/CGPA & academic status.

📦 Deliverables

- GPA calculation service (SGPA + CGPA)
- Retake rules support
- Exclude W/I/P grades
- Academic status update engine
- Admin button to run the engine

🛠 Tasks

- `IGpaCalculator` + implementation
- Filter registrations by semester
- Apply retake policy (best grade)
- Update Student.CGPA + AcademicStatus
- Background job execution (`IHostedService` or Hangfire)

🧪 Testing

- Complex GPA calculation tests
- Regression tests for retakes

📝 Documentation

- GPA calculation algorithm
- Control engine process diagram

📌 WEEK 11 — Graduation Project Module + Reporting

🎯 Goals

Implement graduation project + transcripts.

📦 Deliverables

- Upload, approve graduation project
- Graduation eligibility check
- Student transcript reporting endpoint
- Course history & GPA summary

🛠 Tasks

- Commands: `SubmitProject`, `ApproveProject`
- Eligibility logic (required credits + GMAT)
- Transcript Query returns:
  - List of courses + grades
  - SGPA/CGPA
  - Academic status

🧪 Testing

- Transcript integration test
- Graduation eligibility tests

📝 Documentation

- Transcript format
- Graduation requirements checklist

📌 WEEK 12 — Polishing, Security, Deployment & Final Tests

🎯 Goals

Prepare system for production.

📦 Deliverables

- Performance improvements
- API rate limits
- Final security hardening
- Full deployment on server/Azure/AWS
- CI/CD pipelines
- Documentation complete

🛠 Tasks

- Add caching for heavy queries
- Add response compression
- Add rate limiting (ASP.NET middleware)
- Add health checks
- Final Swagger polishing
- Docker/K8s deployment
- Seed initial production data

🧪 Testing

- Load testing (JMeter / k6)
- Full system integration tests
- Security penetration testing (JWT, roles)

📝 Documentation

- Complete system doc
- Final API Reference
- Deployment guide
- Troubleshooting guide